and plug it into (25):

$$\begin{aligned}(25) \leq & \exp\left(\frac{(2\gamma_\lambda^2+5)(\frac{(8+2\sqrt{14})|\mathcal{I}_2|\Delta}{2\gamma_\lambda^2+5})^2}{8|\mathcal{I}_2|} - \frac{(16+4\sqrt{14})|\mathcal{I}_2|\Delta}{2\gamma_\lambda^2+5}\Delta\right) \\ = & \exp\left(\frac{(15+4\sqrt{14})|\mathcal{I}_2|\Delta^2}{(2\gamma_\lambda^2+5)} - \frac{(16+4\sqrt{14})}{2\gamma_\lambda^2+5}|\mathcal{I}_2|\Delta^2\right) \\ = & \exp\left(-\frac{1|\mathcal{I}_2|\Delta^2}{2\gamma_\lambda^2+5}\right) \\ \leq & \exp\left(-\log\left(\frac{1}{\delta}\right)\right) \\ = & \delta\end{aligned} \tag{31}$$

That is, we just showed that:

$$1-\delta \leq \mathbb{P}(\hat{\alpha}(\tau(\alpha+\Delta)+\varepsilon) > \alpha \mid \mathcal{I}_1) \leq \mathbb{P}(\hat{\tau} < \tau(\alpha+\Delta)+\varepsilon \mid \mathcal{I}_1). \tag{32}$$

The above equation holds for every $\varepsilon > 0$, and thus by taking $\varepsilon \to 0$, and following the continuity of the probability measure, we obtain that $1-\delta \leq \mathbb{P}(\hat{\tau} \leq \tau(\alpha+\Delta) \mid \mathcal{I}_1)$, as required. □

Since the proposed methods construct censoring times that satisfy the conditional independence assumption of Theorem D.1, their validity follow immediately from this theorem with the corresponding definitions of the censoring times and the weights.

E Experimental Details

This section provides additional information about the experimental setup and methodology described in Section 6. Section E.1 details the synthetic experiments, including data generation and model architecture. Section E.2 presents experiments conducted on real-world data. Finally, Section E.3 outlines implementation details that are shared across both experimental settings.

E.1 Synthetic Experiments

Synthetic Covariate Generation. As described in Section 6.2 of the main text, we use synthetic data to evaluate the informativity and coverage of the LPBs constructed by our calibration procedures. Each synthetic data point consists of covariates $X_i \in \mathbb{R}^d$ with $d = 10$, and a prompt-specific unsafe-output probability p_i. The dataset is designed to simulate a setting in which most prompts have a high likelihood of producing unsafe outputs, while a smaller subset is relatively safe. Notably, our experiments show that fitting a model on this synthetic dataset results in a large prediction error, which turns the calibration step more challenging.

To this end, we sample the covariates from a Gaussian distribution:

$$X_i \mid p_i \sim \mathcal{N}\left(\mu(p_i), \sigma^2 I_d\right), \quad \text{with } \sigma = 0.1. \tag{33}$$

where the mean vector $\mu(p_i) \in \mathbb{R}^d$ is designed as follows. First, we construct a pool of p_i values, where (i) 90% of which are drawn uniformly in the range $\log_{10}(p_i) \sim \mathcal{U}[-4,-3]$, and (ii) the remaining 10% are drawn uniformly in the range $\log_{10}(p_i) \sim \mathcal{U}[-6,-5]$. We then assign each p_i to a data point by sampling without replacement from this pool. Then, we define d quantile levels as follows

$$\tau_j = 0.1 + 0.8\,\frac{j-1}{d-1} \quad j = 1,\ldots,d.$$

Next, for a given probability p_i, we compute:

$$\tilde{\mu}(p_i) = \left[F^{-1}_{\text{Geom}(p_i)}(\tau_1)^{1/4}, \ldots, F^{-1}_{\text{Geom}(p_i)}(\tau_d)^{1/4}\right]^\top,$$

where $F^{-1}_{\mathrm{Geom}(p_i)}$ is the quantile function of the Geometric distribution with success probability p_i. The 1/4 power transformation reduces the size of the often extremely high raw quantiles. Lastly, to improve numerical stability, we normalize this vector by the average magnitude across all training examples:

$$\bar{\mu} = \frac{1}{n\,d}\sum_{i=1}^{n}\sum_{j=1}^{d} F^{-1}_{\mathrm{Geom}(p_i)}(\tau_j)^{1/4}, \quad \mu(p_i) = \frac{\tilde{\mu}(p_i)}{\bar{\mu}}.$$

Model Architecture and Training. To estimate the unsafe generation probabilities, we train a neural network with four hidden layers of size 32 each. We used ReLU for non-linearity and a sigmoid for output. We optimize the BCE loss (see Section E.3) using AdamW with learning rate 10^{-4}, weight decay 10^{-5}, batch size 100, for 10 epochs.

Calibration Method Parameters. In all experiments, unless stated otherwise, we used the prior quantile $\tau_{\text{prior}} = 10^{-1/4}$, with trimming threshold M set so that $\gamma = \max(|\mathcal{I}_2| \cdot M/B, 1) = 10$.

E.1.1 Additional Experiments

To confirm that our findings on synthetic data do not depend on a particular choice of calibration/test split, we repeated the experiment from Section 6.2 for 20 independent splits of calibration and test sets. Specifically, in each trial, we fixed the original training set of 45,000 prompts, then randomly partitioned the remaining pool into a calibration set of size 45,000 and a test set of size 10,000. All other settings follow the experimental setup described in Section E.1.

We repeated the experiment for the following values of average sampling budget per prompt $B/|I_2|$: $10, 25, 50, 100, 200, 300, 600, 1200$. In all experiments, we used the same prior quantile $\tau_{\text{prior}} = 10^{-1/4}$, with trimming threshold M set so that $\gamma = \max(|\mathcal{I}_2| \cdot M/B, 1) = 10$, as in Section 6.2.

Figure 3 presents the performance of the methods introduced in this work for 20 data splits. Across every budget level, the performances closely match those from our original fixed-split experiment from Figure 1 in Section 6.2. Next, we evaluate how the target coverage level affects the constructed LPBs and their informativeness by

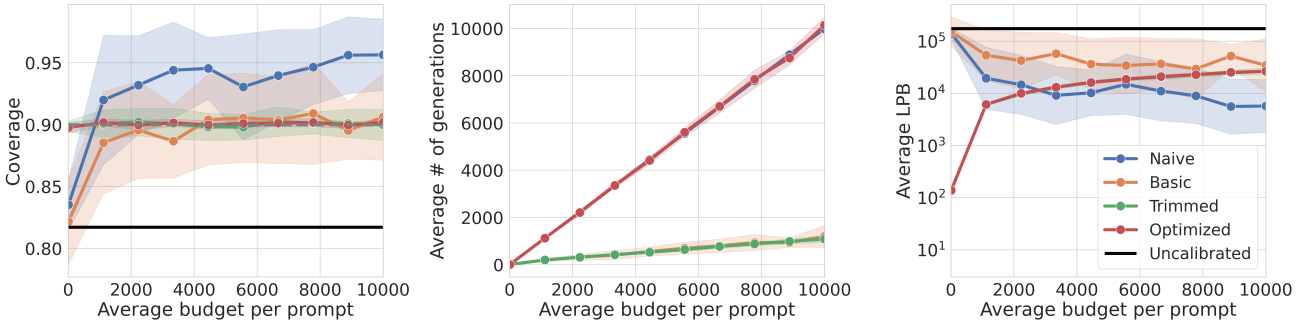


Figure 3: Results of synthetic experiments as a function of average budget per prompt $B/|\mathcal{I}_2|$. **Left:** Coverage (target 90%; gray dashed line). **Center:** Mean number of samplings generated per prompt. **Right:** Mean LPB. Shaded regions represent the standard deviation over 20 runs.

repeating the experiment from Section 6.2 across different nominal coverage levels. We present the results in Figure 4. This figure shows that the `Optimized` consistently achieves its target coverage across all levels.

Finally, we turn to study the effect of the maximal weight size $\gamma = \max(|\mathcal{I}_2| \cdot M/B, 1)$ on the stability and power of the `Trimmed` and `Optimized` calibration methods. Figure 5 compares the two methods under a fixed per-prompt budget of $B/|\mathcal{I}_2| = 1000$. For each weight size, we set the threshold $M = B\gamma/|\mathcal{I}_2|$. As can be seen in this figure, the size of the LPBs we construct increases with γ. However, this comes at the cost of increasing the coverage variance. Observe how the variance of `Optimized` method for $\gamma = 100$ is similar to that of `Trimmed` method for $\gamma = 10$, but the former yields much more informative LPBs under these corresponding γ values. This is a result of the variance-reducing optimization objective in (10).

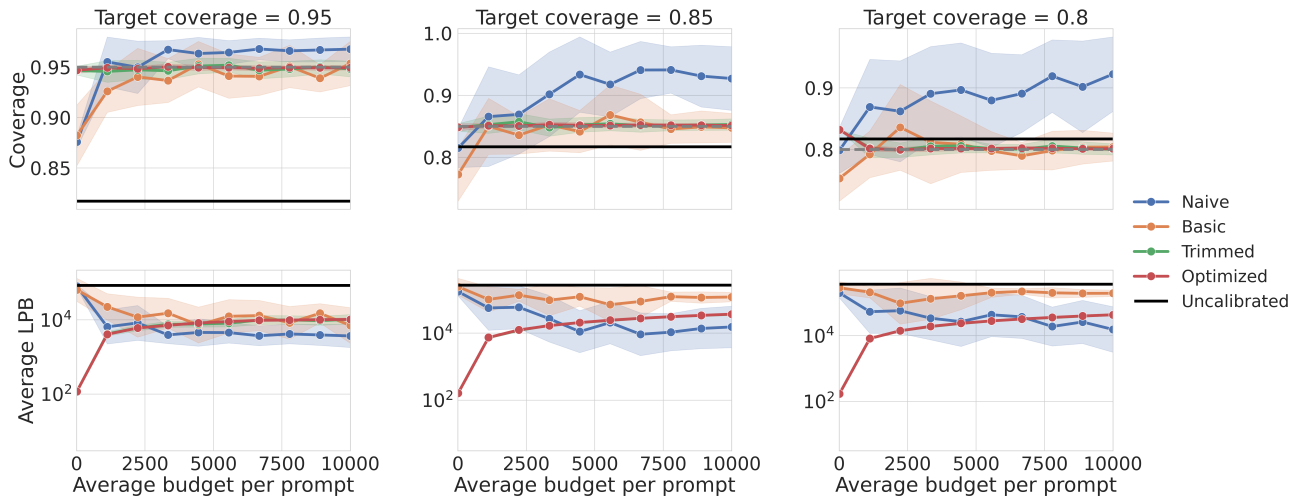


Figure 4: Results of synthetic experiments as a function of average budget per prompt $B/|\mathcal{I}_2|$. **Top:** Coverage (target level indicated by a gray dashed line). **Bottom:** Mean LPB. Shaded regions represent the standard deviation over 20 runs.

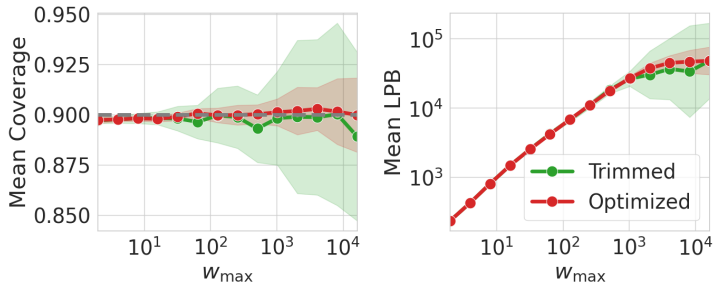


Figure 5: Results of synthetic experiments as a function of the maximum weight $w_{\max}$ of the `Trimmed` and `Optimized` methods. **Left**: Coverage (target 90%; gray dashed line). **Right**: Average LPB. Shaded regions present the standard deviation across 20 runs.

We compare the proposed `Optimized` method with an oracle quantile regression model on our synthetic dataset. The oracle model outputs the true conditional quantile of $T \mid X$ as the LPB, and therefore achieves the target coverage rate of $1 - \alpha = 90\%$ by definition. These oracle LPBs are also the most informative possible by construction. Figure 6 reports the performance of the `Uncalibrated` quantile regression model, `Naive`, `Basic`, `Trimmed`, `Optimized`, and the oracle quantile regression model. As expected, the oracle model achieves the nominal coverage level. Notably, our `Optimized` method produces LPBs of comparable size to the oracle as the budget increases, demonstrating its efficiency in approaching the best achievable performance even without access to the oracle quantiles.

We compare our approach with a calibration method that has an infinite budget on our synthetic data. With an unlimited budget, we can observe an unsafe event for every calibration sample and thus can employ classic conformal prediction (Vovk et al., 2005; Papadopoulos, 2008) without data re-weighting to obtain LPBs that achieve $1 - \alpha = 90\%$ coverage rate. Specifically, we estimate the miscoverage rate similarly to (6), but here, we use all calibration points:

$$\hat{\alpha}^{\text{ib}}(\tau) = \frac{1}{|\mathcal{I}_2| + 1} \sum_{i \in \mathcal{I}_2} \mathbb{I}\{T_i < \hat{q}_\tau(X_i)\} \tag{34}$$

and construct the LPB as in (7) with the above $\hat{\alpha}^{\text{ib}}$.

We summarize the performance of all methods in Figure 7. The results reveal that the infinite budget method achieves the desired coverage rate with lower variability than the alternatives. Moreover, our `Optimized` ap-

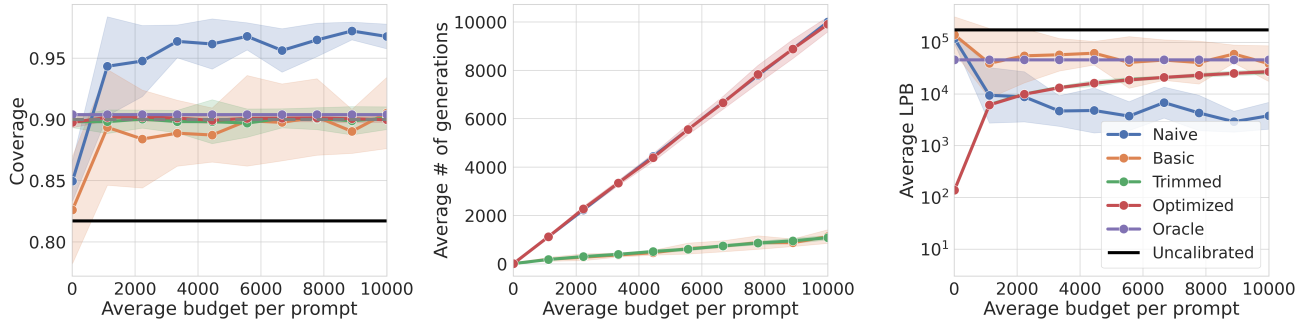


Figure 6: Synthetic data experiment with an oracle quantile regression model. **Top:** Coverage (target 90%%; gray dashed line). **Bottom:** Mean LPB. Shaded regions represent the standard deviation over 20 runs.

proach produces LPBs comparable in size to those of the infinite-budget calibration as the budget increases, demonstrating its effectiveness in approaching optimal performance even under budget constraints.

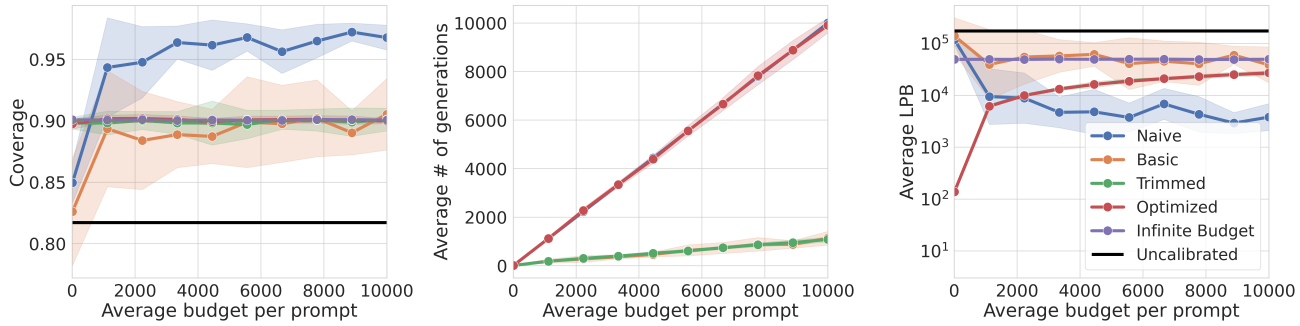


Figure 7: Synthetic data experiment with a calibration method that has an infinite budget. **Top:** Coverage (target 90%; gray dashed line). **Bottom:** Mean LPB. Shaded regions represent the standard deviation over 20 runs.

E.2 Real-Data Experiments

Calibration Method Parameters. In all experiments, unless stated otherwise, we used the prior quantile $\tau_{\text{prior}} = 10^{-1/4}$, with trimming threshold M set so that $\gamma = \max(|\mathcal{I}_2| \cdot M/B, 1) = 2$.

Efficient Prompt-Parallel Sampling. To efficiently generate outputs during our calibration procedures, we implement a prompt-parallel generation algorithm on top of the vLLM package (Kwon et al., 2023). In each iteration, we gather a batch of 1500 prompts X_i along with their corresponding censoring budgets C_i, and distribute them across multiple GPUs. The pipeline iteratively performs the following steps in parallel for a batch of prompts, until all prompts in the calibration set are processed:

1. Generate an output $\mathcal{G}(X_i)$ using the LLM.
2. Apply the audit function $\text{Audit}\big(X_i, \mathcal{G}(X_i)\big)$ to identify unsafe outputs.
3. Check whether X_i has either: (i) reached the C_i generations limit; or (ii) produced an output that fails the audit. If at least one of these two conditions is met, remove the prompt from the pipeline and replace it with a new prompt.

By promptly removing any prompt that yields an unsafe output, the processing maintains high throughput and avoids idle GPU time.